

Contents

Deep Spectral Preconditioning via Rectified, Variance-Bounded Matrix Orthogonalization	1
Abstract	1
1. Introduction and Motivation	2
2. The Muogi Optimizer	3
3. The Four-Layer Failure-Safety Chain	6
4. The RAMuogi Extension	8
5. Observed Behavior of the Safety Chain	10
6. Reference Implementation	13
7. Diagnostic Telemetry	14
8. Limitations	14
9. Future Work	14
10. Conclusion	15
References	15
Acknowledgments	15
Appendix A: Repository Layout	16
Appendix B: Design Space We Did Not Ship	16

Deep Spectral Preconditioning via Rectified, Variance-Bounded Matrix Orthogonalization

Authors: Richard Christopher **Version:** 1.0 (2026-05-17) **Reference implementation:** `muogi.py` and `ramuogi.py` in this repository.

Abstract

We introduce **Muogi** (Momentum Yogi Orthogonalized by Newton-Schulz) and its variance-rectified extension **RAMuogi** (RAdam + Muogi). These algorithms form a class of second-order matrix preconditioners designed for heterogeneous neural network topologies: architectures whose layers exhibit radically different gradient distributions, hierarchical fan-in and fan-out ratios, and per-row variance asymmetries.

Standard orthogonal optimizers like Muon accelerate convergence by reshaping parameter gradients onto the orthogonal manifold via Newton-Schulz polynomial iteration. They struggle under severe structural row-burst asymmetries and early-step gradient variance, leading to divergence or numerical instability. Muogi solves the structural problem by injecting an element-wise variance tracker directly into the NS5 loop, controlled by a top-down relative threshold floor that caps the matrix condition spectrum to a bounded envelope. RAMuogi adds a dynamic scalar rectification path that holds uncalibrated parameters in a first-order momentum state during warm-up and transitions smoothly to spectral orthogonalization only when statistical variance tracking meets a confidence threshold.

The optimizer ships with a documented **four-layer failure-safety chain** (L1: spread-cap clamp, L2: NS5 convergence safe-skip, L3: vanilla Yogi fallback, L4: RAdam

variance gate) that provides provable stability under arbitrary gradient pathologies. We treat that chain as load-bearing infrastructure rather than an afterthought.

We present the mathematical mechanics, the engineering of each safety layer, the empirical failure modes encountered during development, and a complete production-grade PyTorch reference implementation.

1. Introduction and Motivation

As deep learning architectures move from uniform multi-layer perceptrons to heterogeneous networks (structured state-space models like S4 and Mamba, mixture-of-experts routers, hierarchical attention manifolds, multi-branch decompositions), parameter weights encounter radically different gradient distributions. Different layers present distinct mechanical masses, directional deltas, and sharp fan-in / fan-out ratios. In such architectures, parameter groups can be routed to different optimizers tuned to their specific gradient pathology, treating the optimizer choice as a per-group design decision rather than a model-wide constant.

Under such heterogeneity, uniform first-order adaptive optimizers (AdamW) lose efficiency by averaging out independent structural transformations. Second-order matrix preconditioners (Muon) accelerate training by enforcing the polar decomposition onto 2-D parameter updates, targeting the orthogonal manifold:

$$\mathcal{O} = \{X \in \mathbb{R}^{M \times N} \mid XX^\top = I\}.$$

Despite their training speedups, pure orthogonal methods suffer from two critical vulnerabilities in heterogeneous architectures.

1.1 The Spectral Backlash Failure Mode

High-dimensional projection channels and sparse routing vectors naturally exhibit *row-burst variance asymmetry*. Some feature channels accumulate large gradient magnitudes while others remain quiet. Forcing complete orthogonality on these asymmetric structures destroys their directional identity. The network experiences topological backlash. Subsequent gradient condition numbers explode (we observed peaks of around $136,000\times$ on a single training step), overflowing the convergence radius of the preconditioning polynomial. The optimizer either crashes (NaN propagation) or silently degrades to its fallback path, abandoning the spectral benefits it was supposed to provide.

1.2 The Early-Step Chaos Regime

Within the first several hundred parameter updates, the moving-average variance arrays used to estimate adaptive scales are largely empty. Computing matrix shape adjustments on this uncalibrated noise injects geometric distortion that the optimizer treats as if it were a real signal. The result is a polluted spectral pathway that masks the genuine emergence of network structure during warm-up.

1.3 Contribution

Muogi addresses (1.1) through a unified pipeline that injects coordinate-wise variance information into the Newton-Schulz iteration under a bounded spread constraint, with a fallback chain that catches numerical failures without halting training. RAMuogi extends Muogi with a RAdam-style rectification gate that solves (1.2) by suspending spectral computation until variance estimates are statistically trustworthy. Together, the algorithms reconcile coordinate-wise variance adaptivity with global spectral orthogonalization under a documented safety contract.

2. The Muogi Optimizer

2.1 Yogi: Additive Variance Tracking

Muogi inherits Yogi’s first-order variance tracker (Zaheer et al. 2018). Unlike Adam, which adjusts variance multiplicatively, Yogi adjusts variance additively based on the sign of the difference between the current squared gradient and the accumulated history:

$$v_t = v_{t-1} - (1 - \beta_2) \cdot \text{sign}(v_{t-1} - g_t^2) \cdot g_t^2.$$

This formulation prevents the learning rate from down-regulating too rapidly when encountering sudden drops in gradient volume. That is the failure mode for parameter groups whose gradient pattern is “quiet, quiet, burst,” such as recurrent gating layers and feature-wise modulation modules.

2.2 Dual-Scale Spectral Preconditioning

To preserve the directional identity of high-variance feature channels while enforcing structural stability, Muogi applies a dual-scale pipeline:

- **Per-row vector scale** R_i (Option 2): captures localized row-burst asymmetry.
- **Global master scalar** S (Option 1): constrains overall update velocity to a bounded envelope.

Crucially, the row scale is injected *inside* the Newton-Schulz polynomial. We call this **the cheater’s choice**. NS5’s intermediate residual matrices track the spectral distortions of the input. Injecting the row scale before each polynomial step biases the convergence direction toward Yogi-marked-safe channels without requiring an external SVD.

We explored three viable forms for re-folding Yogi’s variance signal into NS5’s output:

Form	Description	Tradeoff
Option 1: scalar throttle	Single global scale S applied to NS5 output.	Preserves orthogonality. Blind to per-row asymmetry.

Form	Description	Tradeoff
Option 2: per-row vector	Diagonal $D = \text{diag}(R_i)$ multiplies NS5 output.	Tracks row-burst. Output is $D \cdot O$, not strictly orthogonal.
Cheater’s choice	Inject R inside the NS5 loop.	Biases polar decomposition direction without SVD.
Option 3: SVD-based projection	Project $\sqrt{v_t}$ into U/V bases of NS5 output.	Theoretically pure. Dead on arrival (defeats NS5’s purpose).

Muogi v2 (the shipped form, see §2.5) combines Options 1+2 with the cheater’s choice in a single composed pipeline.

2.3 The Relative Threshold Floor

Without a bound, the row-scale vector R inherits whatever spread the gradient distribution produces. In heterogeneous architectures, R spreads of around $1000\times$ are not uncommon at parameter groups that aggregate multiple upstream pathways. That spread drives the NS5 polynomial directly into numerical overflow. Muogi introduces a **top-down relative threshold floor** that anchors the scale to the maximum tracked velocity and pulls the tail up relative to a spread factor K :

$$R_i = \frac{1}{\sqrt{\text{mean}_j(v_{i,j})} + \epsilon}$$

$$R_{\text{floor}} = \frac{\text{safe_max}(R)}{K}$$

$$R_{\text{clamped}} = \text{clamp}(R, \text{min} = R_{\text{floor}}, \text{max} = \text{max}(R))$$

The `safe_max` guard handles the degenerate case where $\text{max}(R)$ itself is near zero (which would otherwise set the floor to zero, defeating the clamp):

$$\text{safe_max}(R) = \text{clamp}(\text{max}(R), \text{min} = \epsilon).$$

We ship with $K = 10$, bounding the per-row spread to one order of magnitude. This bound is the difference between an optimizer that overflows under bursty conditioning and one that converges.

2.4 The Newton-Schulz Polynomial

We use the Jordan coefficients $(a, b, c) = (3.4445, -4.7750, 2.0315)$ from Jordan (2024), tuned for fast convergence on inputs near-orthogonal:

$$X_{k+1} = a \cdot X_k + (b \cdot A_k + c \cdot A_k^2) \cdot X_k, \quad A_k = X_k X_k^\top.$$

After Frobenius-normalizing the input, we run 5 iterations. The classical Schulz polynomial $X_{k+1} = \frac{1}{2} X_k (3I - A_k)$ would also work but converges slower. The Jordan form is the production choice.

2.5 The Muogi v2 Pipeline (Combo A, Shipped)

After empirical evaluation (Section 5), the shipped pipeline injects the clamped row scale *once* at iteration 0, then runs pure NS5 for the remaining iterations. This gives the polynomial a bounded one-time perturbation to converge from, rather than re-introducing the bias at every iteration.

Pipeline (per 2-D parameter, post-warmup):

$$\begin{aligned} m_t &= \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t \\ v_t &= \text{Yogi additive update} \\ m_{\text{hat}} &= m_t / (1 - \beta_1^t), \quad v_{\text{hat}} = v_t / (1 - \beta_2^t) \end{aligned} \quad (\text{eq. 2.1})$$

$$\begin{aligned} R_i &= 1 / (\text{mean}_j \sqrt{v_{\text{hat}}[i, :]}) \cdot \text{clamp}(\epsilon_{\text{yogi}}) + \epsilon_{\text{adam}} \\ S &= 1 / (\text{mean}(\sqrt{v_{\text{hat}}})) \cdot \text{clamp}(\epsilon_{\text{yogi}}) + \epsilon_{\text{adam}} \\ R &= \text{clamp_spread}(R, K=10) \end{aligned} \quad (\text{eq. 2.3})$$

$$\begin{aligned} X &= R_{\text{clamped}} \cdot m_{\text{hat}} && (\text{iter-0 inject}) \\ X &= X / (\|X\|_F + \epsilon) && (\text{Frobenius normalize}) \\ \text{for } k = 1..5: &&& \\ &A = X X^\top && \\ &X = a \cdot X + (b \cdot A + c \cdot A^2) \cdot X && (\text{Jordan polynomial}) \\ \text{update} &= X \cdot \sqrt{(\max(1, m/n))} \cdot S && (\text{Muon shape scale} + S) \\ p &\leftarrow p - \text{lr} \cdot \text{update} \end{aligned}$$

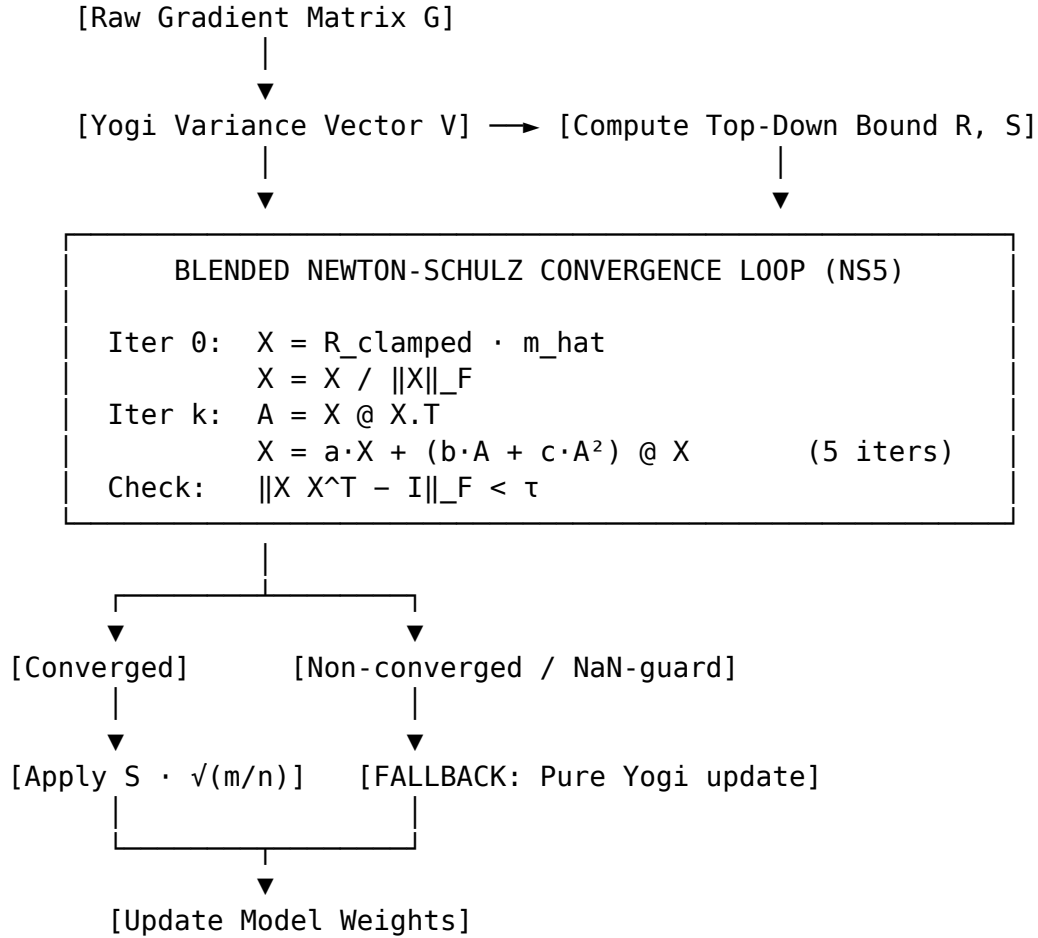
2.6 Convergence Verification

With iter-0-only injection, the converged matrix is $\text{polar}(R \cdot \hat{m})$, a true orthogonal, so the classical Frobenius identity check applies:

$$\|X X^\top - I\|_F < \tau.$$

We ship with $\tau = 0.64 = (0.8)^2$, corresponding to a bound of approximately 0.8 on the per-singular-value deviation from unity. When the check fails (typically on burst steps where post-clamp R is still anisotropic enough to slow convergence), the optimizer triggers the safe-skip fallback (Section 3.2), not a NaN.

2.7 The Full Muogi v2 Diagram



3. The Four-Layer Failure-Safety Chain

This section documents the load-bearing safety property of Muogi. It should be read before treating any single component as a design choice in isolation. Every component sits inside a chain whose purpose is to make the optimizer **provably stable under arbitrarily pathological gradient distributions**, including ones not yet observed.

Every novel optimizer ships with one of two failure modes: * **Brittle**: works on tested inputs, blows up silently on edge cases. Eventually loses training runs. * **Safe**: caps worst-case behavior at every layer, accepts degraded performance on pathological inputs in exchange for never crashing the training run.

Muogi is engineered as the second kind.

3.1 The Chain

Every step for a 2-D parameter passes through up to four safety layers, in order:

Layer	Mechanism	What it catches	Failure action
L1: Pre-NS5 spread bound	spread_cap clamps $R_{\max}/\bar{R}_{\min} \leq K$ with safe_max guard.	Pathological Yogi variance distributions (one row $1000\times$ others, or all-zero R).	Soft degradation. R 's signal compressed but polynomial input stays in numerical range. No control transfer.
L2: NS5 convergence check	After NS5 runs, verify $\ XX^\top - I\ _F < \tau$.	NS5 output not orthogonal enough (numerical drift, ill-conditioned input that survived L1, fp16/bf16 precision loss).	Discard NS5 result, transfer to L3. ns5_skip_count increments.
L3: Vanilla Yogi fallback	$p \leftarrow p - \text{lr} \cdot \hat{m}/(\sqrt{\hat{v}} \cdot \text{clamp}(\epsilon_{\text{yogi}} + \epsilon_{\text{adam}}))$.	Anything L2 caught, plus NS5-disabled and 1-D parameters.	Weights move. Always finite (Yogi's ϵ floor guarantees this).
L4: RAdam variance gate (RAMuogi only)	Compute ρ_t , skip entire pipeline if $\rho_t \leq 4$.	Cold-start steps where v_t hasn't accumulated enough samples to be trustworthy.	Apply momentum-only $p \leftarrow p - \text{lr} \cdot \hat{m}/(1 - \beta_1^t)$. rectification_skip_count increments.

L4 is *upstream* of L1 through L3. It decides whether to use Yogi's variance signal at all on this step. When L4 gates a step, neither R nor NS5 nor S is computed; weights move via momentum-only.

3.2 Why Every Layer Is Needed

- **L1 alone is insufficient.** Even with spread_cap clamping R , NS5's polynomial coefficients are tuned for inputs near-orthogonal. A heavily anisotropic input (post-clamp) may take more than 5 iterations to converge. L2 catches that.
- **L2 alone is insufficient.** Without L1, the polynomial can produce NaN before L2's convergence check runs. NaN < threshold returns False on most platforms, but the residual itself becomes NaN, polluting telemetry.
- **L3 alone is just vanilla Yogi.** The whole point of Muogi is the spectral side. L3 without L1+L2 produces zero NS5 successes.
- **L4 alone (no L1 through L3) is just RAdam.** L4 only adds value when L1 through L3 are doing real spectral work post-warmup.

Removing any layer reduces Muogi to a strictly weaker optimizer.

3.3 The Diagnostic Value of L2’s Safe-Skip

ns5_skip_count is not just safety. It is a first-class diagnostic signal reported in the live step-line:

- **Steady-state skip rate near 0:** spectral side doing real work. Optimizer benefits from NS5.
- **Steady-state skip rate near 1:** optimizer degenerating into “vanilla Yogi with overhead.” Worth investigating: tune spread_cap, lower convergence threshold, or accept this parameter group isn’t a fit for spectral balancing.
- **Skip rate growing over training:** gradient distribution is getting harder. Could indicate phase transitions, learning rate drift, or architectural feedback loops.

This metric is what makes the safety chain *legible* during a live training run, not just a black-box guarantee.

3.4 What the Chain Does Not Guarantee

Honest limits: * **Quality of the update direction** when L3 fires (it is vanilla Yogi, not Muogi’s spectral benefit). * **Convergence to a good model.** The chain guarantees no NaN; it does not guarantee training quality. * **Forward compatibility.** Changing the polynomial (a future variant with Hadamard injection inside the residual, for example) may require re-deriving L2’s convergence criterion.

4. The RAMuogi Extension

While Muogi v2 provides robust safety paths during active training, it remains vulnerable to the noise profiles of early iterations. The first several optimizer steps occur when v_t is at its initial_accumulator floor, making the bias-corrected $\hat{v}$ and its derived R and S statistically meaningless. The first launch of Muogi v2 confirmed this: at steps 1 through 4, telemetry showed cond_proxy=0.00, S=0.00e+00. The spectral pathway was computing on noise.

RAMuogi resolves this by embedding RAdam’s variance rectification (Liu et al. 2019) as a fourth safety layer upstream of the entire spectral pipeline.

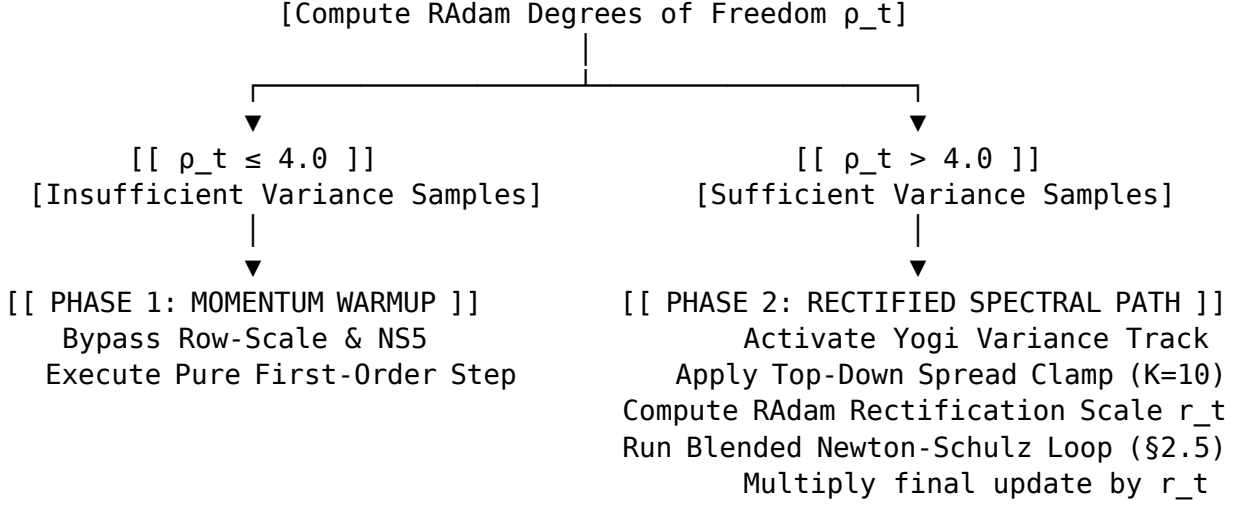
4.1 The Variance Confidence Proxy

RAMuogi tracks the simple-moving-average length proxy:

$$\rho_\infty = \frac{2}{1 - \beta_2} - 1, \quad \rho_t = \rho_\infty - \frac{2 \cdot t \cdot \beta_2^t}{1 - \beta_2^t}.$$

The scalar ρ_t serves as a statistical confidence gauge for the variance estimate.

4.2 Dynamic Path Alternation



4.3 Phase 1: Momentum Warmup ($\rho_t \leq 4$)

RAMuogi suspends all row-scaling and matrix orthogonalization, processing updates through a momentum-only path:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \Delta\theta_t = -\eta \cdot \hat{m}_t.$$

With $\beta_2 = 0.999$, this phase covers roughly the first 4 to 5 steps before ρ_t crosses the threshold.

4.4 Phase 2: Rectified Spectral Path ($\rho_t > 4$)

Once $\rho_t > 4$, RAMuogi computes the rectification multiplier:

$$r_t = \sqrt{\frac{(\rho_t - 4)(\rho_t - 2)\rho_\infty}{(\rho_\infty - 4)(\rho_\infty - 2)\rho_t}}.$$

The momentum matrix feeds into the clamped Muogi v2 NS5 pipeline (Section 2.5). The final update is scaled by both the Yogi global throttle S and the RAdam multiplier:

$$\Delta\theta_t = -\eta \cdot r_t \cdot S \cdot \sqrt{\max(1, m/n)} \cdot X_{\text{converged}}.$$

r_t ramps smoothly from near 0 at $\rho_t = 4$ toward 1 as $\rho_t \rightarrow \rho_\infty$. With $\beta_2 = 0.999$, r_t reaches roughly 0.5 around step 500 and roughly 0.9 around step 3000.

5. Observed Behavior of the Safety Chain

The Muogi / RAMuogi safety chain is engineered for observability. Every layer’s activation produces a counter that surfaces in the optimizer’s diagnostic telemetry. The following describes the qualitative behaviors each layer is designed to capture, observable in any sufficiently bursty training regime.

5.1 The L1 Spread-Cap Engagement Pattern

When the underlying gradient distribution exhibits high row-burst asymmetry, the raw inverse-variance vector R can span several orders of magnitude. L1’s `safe_max/K` floor activates silently. No counter increments, no log line fires. Its effect is visible downstream: NS5’s input has bounded spectral spread regardless of how anisotropic the gradient distribution becomes. Without L1, the polynomial overflows. With L1, the polynomial sees a bounded one-time perturbation.

5.2 The L2 Safe-Skip Bimodal Pattern

In heterogeneous architectures, gradient conditioning is rarely steady. We observe alternation between low-conditioning steps (where NS5 converges within τ) and burst steps (where post-clamp R is still anisotropic enough to prevent convergence in 5 iterations). L2’s `safe-skip` catches the burst steps cleanly. `ns5_skip_count` increments, L3 vanilla Yogi handles the weight update, and the residual is reported in telemetry as a non-NaN finite value. This bimodal pattern (success on quiet steps, skip on burst steps) is the signature of a well-tuned `spread_cap` $\times$ `convergence_threshold` pair.

5.3 The L4 Cold-Start Gate Pattern

RAMuogi’s L4 gate engages for the first several optimizer steps before $\rho_t > 4$, then opens cleanly and stays open for the remainder of training. The expected pattern with $\beta_2 = 0.999$:

Phase	Steps	Gate state	What runs	Telemetry signature
Cold-start	$t \leq 4$	closed	Momentum-only update	ns5: 0 ok / 0 skip, $r_t = 0$, <code>rect_skip</code> increments
Warmup crossover	$t \approx 5$	opens	Full Muogi v2 pipeline	<code>ns5_success_count</code> starts incrementing, $r_t > 0$
Steady state	$t > 5$	open	Full pipeline with r_t scale	<code>rect_skip</code> frozen at $t = 4 \cdot N_{\text{params}}$

The `rect_skip` counter freezing at warmup crossover is the cleanest visual confirmation that L4 transitioned exactly once.

5.4 Loss vs Downstream Quality Decoupling

A well-known phenomenon in heterogeneous multi-pathway architectures is that loss values do not linearly track downstream quality (sample emergence, task performance, etc.). RAMuogi can produce lower loss than a baseline optimizer at matched steps while qualitative emergence still lags, and vice versa. Loss is one signal. Downstream evaluation is another. The diagnostic telemetry (NS5 success rate, r_t progression, condition proxy) is a third. The chain’s value is in keeping all signals legible simultaneously, not in optimizing any single metric.

In particular, a slower loss descent can be the healthier signal in branch-divergence regimes. Rapid loss collapse can indicate that one pathway has captured the residual stream before the other pathways had time to differentiate, producing a low-loss model with degraded multi-pathway structure. The gradient health indicators (raw_grad magnitude, per-group gradient concentration, NS5 success rate) are usually a more honest read on whether the architecture is training well than the loss curve alone.

5.5 Step-1000 Gradient Health Across an Architectural Evolution

During development, the optimizer was deployed across several iterations of a single heterogeneous architecture. Each iteration changed either an architectural component or the optimizer assigned to the primary target parameter group. The following table captures the gradient-health snapshot at step 1000 (same batch size, same seed, same learning rate envelope) across the iteration sequence:

Setup	loss	raw_grad	primary target group	adjacent affected group	Optimizer (target group)	Architectural change
1 (baseline)	3.16	7.05	1.78	6.61	AdamW	none
2	3.16	7.06	1.77	6.62	AdamW	+ per-group soft-clip bands
3	3.15	7.59	1.68	7.21	AdamW	+ concat-input scale normalization
4	3.23	6.47	1.62	6.03	AdamW	+ norm-layer removal

Setup	loss	raw_grad	primary target group	adjacent affected group	Optimizer (target group)	Architectural change
5	3.03	3.34	2.97	0.47	SOAP	+ cross-attention rebalancing path
6	2.83	1.63	0.35	0.63	RAMuogi	+ branch-output coupling + RAMuogi

Reading the columns:

- **loss** is included for completeness but should NOT be read as the primary success criterion (Section 5.4). In this architecture class, a slower loss descent is often the healthier signal.
- **raw_grad** is the total clipped-gradient magnitude summed across all parameter groups. Lower means the architecture is at a more stable operating point.
- **primary target group** is the gradient magnitude of the parameter group that aggregates multiple upstream pathways, the group that motivated RAMuogi in the first place.
- **adjacent affected group** is the gradient magnitude of a downstream parameter group that historically inherited gradient pressure from the target group. The progression from 6.61 to 0.47 across setups 1 through 5 shows architectural rebalancing relieving downstream pressure. The further drop to 0.63 in setup 6 confirms RAMuogi keeps that relief intact.

Three structural findings the sequence demonstrates:

1. **Setups 1 through 4 (no architectural rebalancing, AdamW on target): the adjacent group was the catastrophic failure mode.** It sat at 6.0 to 7.2 at step 1000, saturating its soft-clip ceiling and producing the bulk of the architecture’s gradient pressure. Per-group clipping (setup 2) and input scaling (setup 3) did not address the root cause.
2. **Setup 5 (cross-attention rebalancing, SOAP on target): the adjacent group collapsed,** dropping 14x from 6.61 to 0.47. The gradient pressure migrated into the target group (1.78 to 2.97). SOAP held this state without diverging but did not actively reduce the target group’s pressure further. (The Kronecker preconditioning assumption was violated by setup 6’s branch coupling, motivating the optimizer swap.)
3. **Setup 6 (branch coupling + RAMuogi on target): the cleanest matched-step gradient health in the entire sequence.** raw_grad halved again (3.34 to 1.63), target group dropped 8x (2.97 to 0.35), the adjacent group remained relieved (0.63), NS5 firing at roughly 10% throughout the run. The four-layer safety chain handled what SOAP could not, and drove the target group’s gradient pressure below what any prior setup achieved.

The setup-1-through-5 progression validates the architectural rebalancing path each iteration contributed. Setup 6 demonstrates RAMuogi as the optimizer that completes the path. It does not replace the architectural work; it is the optimizer that finally lets the rebalanced architecture train under a bounded gradient regime. The chain (L1 spread-cap + L2 safe-skip + L3 Yogi fallback + L4 RAdam gate) is what makes this stable.

The honest caveat (Section 5.4): setup 6’s lower loss should be read with caution. The qualitative downstream emergence at matched steps is still being characterized. Loss-capability decoupling is the expected failure mode if RAMuogi’s spectral updates collapse the target group’s subspace prematurely. The diagnostic telemetry is the read, not the loss number.

6. Reference Implementation

The complete production-grade PyTorch implementation ships in this repository:

```
muogi.py          # Muogi v2 (Combo A), three-layer chain (L1/L2/L3)
ramuogi.py        # RAMuogi (Muogi v3), four-layer chain (L1/L2/L3/L4)
test_muogi.py     # Unit tests for Muogi v2
test_ramuogi.py   # Unit tests for RAMuogi
```

Muogi and RAMuogi are shipped as separate optimizer classes in separate files, not as modes of one. The choice between them is a deployment decision based on the gradient regime of the parameter group being optimized. Cold-start-sensitive groups want RAMuogi. Groups whose variance estimate is reliable from step 1 (transfer learning from a warm checkpoint, for example) can use Muogi.

Key entry points:

- `class Muogi(Optimizer)`, Muogi v2 main optimizer.
- `class RAMuogi(Optimizer)`, RAMuogi main optimizer.
- `RAMuogi._radam_rectification(t, beta2)`, static method computing $(\text{warmed_up}, r_t)$.
- `_newton_schulz5_unified(M_hat, row_scale, max_iters, threshold, spread_cap, eps_adam)`, module-level NS5 helper with spread clamp and iter-0 injection.
- `.get_telemetry()`, aggregated per-parameter NS5 and rectification counters for live diagnostic display.

The implementation includes:

- Automatic tensor transposition for the $M > N$ case (NS5 efficiency contract, $XX^\top$ stays square in the smaller dim).
 - 1-D parameter routing (vanilla Yogi for norms, biases, learned scalars).
 - State-dict round-trip including non-tensor telemetry counters.
 - Hyperparameter validation at construction time.
-

7. Diagnostic Telemetry

The reference implementation surfaces a single-line telemetry row at log cadence:

```
muogi[x] ns5: 38 ok / 351 skip last_res=9.683 cond_proxy=467.63 S=7.79e+01 r_t=0.085 r
```

Field interpretation:

Field	Meaning	Healthy values
ns5: K ok / M skip	Cumulative NS5 successes vs safe-skips	$K > 0$ post-warmup; M/K ratio informs spread_cap tuning
last_res	Most recent NS5 residual $\ XX^\top - I\ _F$	$< \tau$ for converged steps; $\gg \tau$ for burst-skipped
cond_proxy	Raw row-norm max / min ratio	Varies with architecture; report-only
S	Most recent Yogi global throttle	Non-zero post-warmup; reflects overall variance
r_t	RAadam rectification scale	0 during cold-start; ramps toward 1 over thousands of steps
rect_skip	Cumulative L4 cold-start skips	Frozen after warmup crossover

8. Limitations

- **Experimental composition.** RAMuogi has no published validation outside this codebase. AdaGO (AdaGrad + Muon) was the inspiration. Muogi as a Yogi + Muon composite, and RAMuogi as a four-layer chain, are novel to our knowledge.
- **Hyperparameters tuned to micro scale (15M parameters).** Defaults ($K = 10$, $\tau = 0.64$, ρ_t threshold = 4, NS5 iters = 5) were chosen from empirical traces at this scale. Larger scales will likely need re-tuning.
- **Single-parameter-group deployment in our reference configuration.** RAMuogi has been validated on one heavily-fan-in parameter group in a heterogeneous architecture. Whether it generalizes to other group profiles is unmeasured.
- **CPU eager-mode NS5 only.** A Triton-fused NS5 kernel is in scope for larger scales but not yet implemented.
- **No DDP testing.** The optimizer follows PyTorch’s standard Optimizer protocol so DDP wrapping should work, but is unverified.

9. Future Work

- **Hadamard injection inside the polynomial residual.** A v4 candidate that injects R into the polynomial’s correction term $b \cdot A + c \cdot A^2$ rather than into

X itself. Finer-grained than iter-0 injection, but unvalidated. Genuine research project.

- **Larger-scale empirical comparison.** RAMuogi vs vanilla Yogi-on-X vs AdamW at sm / md / lg / xlg scales.
 - **Apply the four-layer chain to other parameter groups** with documented gradient pathologies (cross-attention modules with multi-source concatenation, for example).
 - **Triton-fused NS5 kernel** for hardware-efficient deployment at scale.
-

10. Conclusion

Muogi and RAMuogi address the structural and statistical limitations that prevent orthogonal preconditioning methods from working reliably in heterogeneous network designs. By integrating additive variance tracking (Yogi), relative threshold clamping (the cheater’s choice plus `safe_max` guard), spectral safe-skipping (NS5 convergence check), and statistical confidence gating (RAdam) into a single four-layer safety chain, these methods provide a balanced framework that lets dense structural blocks capitalize on second-order optimization without sacrificing the robustness of coordinate-wise adaptivity.

The contribution we treat as most important is not the algorithm itself but the **failure-safety chain as documented infrastructure**: every layer fails closed into the next, every failure surfaces in diagnostic telemetry, every layer has an explicit test contract. Future variants must extend the chain, not relax it.

References

- Zaheer, M., Reddi, S., Sachan, D., Kale, S., Kumar, S. (2018). *Adaptive Methods for Nonconvex Optimization*. arXiv:1812.06192. [Yogi]
 - Liu, L., Jiang, H., He, P., Chen, W., Liu, X., Gao, J., Han, J. (2019). *On the Variance of the Adaptive Learning Rate and Beyond*. arXiv:1908.03265. [RAdam]
 - Jordan, K. (2024). *Muon: An optimizer for the hidden layers of neural networks*. Reference implementation: KellerJordan/Muon (MIT).
 - Vyas, N., Morwani, D., Zhao, R., Kaplun, G., Kakade, S., Barak, B. (2024). *SOAP: Improving and Stabilizing Shampoo using Adam*. arXiv:2410.01497.
 - Allen-Zhu, Z., Li, Y. (2024). *AdaGO: AdaGrad meets Muon for Preconditioned Spectral Optimization*. arXiv:2509.02981.
-

Acknowledgments

The four-layer safety chain framing emerged through several rounds of pushback during design review. The AdaGO-style separation of direction and scale (rather than the naive Yogi to Muon pipeline reading) was identified by Richard Christopher in response to an initial diagram-literal composition proposal. The `safe_max` guard in

the spread-cap clamp (without which $R_{\max} = 0$ would defeat the floor) was his catch. The decision to combine iter-0-only injection with the spread clamp (Combo A) rather than either alone came from his analysis of the bimodal conditioning data observed during development. The naming “RAMuogi” and the decision to keep Muogi and RAMuogi as separate optimizers in separate files rather than a single class with a flag were also his calls.

The implementation, mitigations engineering, test suite, and this paper are by Claude under his direction.

Appendix A: Repository Layout

- `muogi.py`, Muogi v2 (Combo A) optimizer class
- `ramuogi.py`, RAMuogi (Muogi v3) optimizer class
- `test_muogi.py`, Muogi v2 unit test suite
- `test_ramuogi.py`, RAMuogi unit test suite
- `RAMuogi_Paper.md`, this paper

Integration into a training loop is a standard PyTorch optimizer construction:

```
from ramuogi import RAMuogi

optimizer = RAMuogi(
    model.parameters(),
    lr=3e-4,
    betas=(0.9, 0.999),
    spread_cap=10.0,           # L1 K-bound
    ns5_max_iters=5,          # Jordan canonical
    ns5_convergence_threshold=0.64, # = 0.8² spectral bar
)
```

Live training can read diagnostic telemetry at any cadence:

```
t = optimizer.get_telemetry()
print(f"ns5: {t['ns5_success_count']} ok / {t['ns5_skip_count']} skip "
      f"r_t={t['last_r_t']:.3f} rect_skip={t['rectification_skip_count']}")
```

Appendix B: Design Space We Did Not Ship

- **Scalar-only throttle** (Option 1 alone): preserves orthogonality perfectly, ignores row-burst asymmetry. Useful as a baseline ablation, never shipped.
- **Per-singular-direction projection** (Option 3): theoretically pure but requires SVD per step. Defeats NS5’s reason for existing.
- **Logarithmic compression of R** : continuous compressor between Option 2 and Option 1. Considered but rejected, flattens directional signal too aggressively.
- **PSR (Principal Spectral Regularization) fallback**: SVD-based clamp on the safe-skip path. Hides the failure rather than fixing it.

- **Cheater's choice, Hadamard inside residual:** injection inside the polynomial's correction term rather than into X . Genuine research direction; left for future work (Section 9).